Lista de Exercícios – Árvores Binárias

Pesquisa e Caminhamento · AEDS 2 (Java) · PUC Minas · baseada no slide do Tópico 4

Árvore de teste: para conferir suas respostas, insira os elementos nesta ordem: `6 3 1 2 5 4 9 8 7 10 11` (a árvore usada como exemplo nos slides).

Parte 1 – Percorrendo a árvore toda

1. Faça um método que retorne a **altura** da árvore. Use a convenção: árvore vazia = –1 e árvore com um nó = 0.
2. Faça um método que retorne a **soma** dos elementos da árvore.
3. Faça um método que retorne o **número de elementos pares** da árvore.
4. Faça um método que retorne `true` se a árvore tiver algum número **divisível por 11**.
5. Faça um método que retorne o **número de folhas** da árvore. Depois, **prove por indução** que, em qualquer árvore binária não vazia, o número de folhas é igual ao número de nós com dois filhos mais um.

Parte 2 – Comparando e transformando árvores

6. Faça um método **estático** que receba dois objetos do tipo árvore binária e retorne um `boolean` dizendo se as duas árvores são **iguais**.
7. Faça um método que transforme a árvore em sua **imagem espelhada**, ou seja, que troque a esquerda pela direita em todos os nós. O que o caminhamento central da árvore espelhada imprime?
8. Faça um método que retorne `true` se uma árvore binária qualquer (não necessariamente construída pelo `inserir`) for uma **árvore binária de pesquisa (ABB)**.

Parte 3 – Usando a propriedade da ABB

9. Faça um método que retorne o **k-ésimo menor** elemento da árvore (k = 1 é o menor).
10. Em uma ABB, faça um método que receba dois elementos a e b presentes na árvore e retorne o **menor ancestral comum** deles, isto é, o nó mais profundo que tem a e b na sua subárvore.

Parte 4 – Aplicações

11. Implemente o **TreeSort**: insira os elementos de um array numa árvore binária e use um caminhamento para devolvê-los ordenados. O array pode ter **elementos repetidos**. Faça também a **análise de complexidade** (melhor, pior e caso médio).
12. Faça o método `No toAB(Celula p1, CelulaDupla p2)`. Ele recebe o nó cabeça de uma lista simples e o de uma lista dupla, cria uma árvore binária com os elementos **intercalados** das duas listas e retorna o endereço da raiz.

13 Insira **100.000 elementos distintos** aleatórios na árvore. A cada 1.000 inserções, mostre na tela: o número de elementos da árvore; o lg (log na base 2) desse número; o número médio de comparações de uma pesquisa com sucesso, **medido** e **teórico**; e a altura da árvore.